

LABORATORY RECORD

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	4
Student Name	K. Anusha
Roll No.	A24126552088
Date	29-08-2026

1. TITLE

JavaScript Basics – Arrays, Functions, and Object-Oriented Concepts (Encapsulation & Inheritance)

2. AIM

To understand and implement the basic concepts of JavaScript arrays, functions, arrow functions, and object-oriented programming concepts such as encapsulation and inheritance by creating simple programs and observing their output.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of arrays in JavaScript.
- To create and initialize arrays with values.
- To access and modify array elements using index numbers.
- To determine the length of an array.
- To use array methods such as push(), pop(), shift(), and unshift().
- To create and use functions, including arrow functions, in JavaScript.
- To pass arrays as arguments to functions.
- To write functions that find the highest and lowest values in an array.
- To understand encapsulation using private class fields (#) in JavaScript.
- To understand inheritance using the extends and super keywords.

4. THEORY

JavaScript is a high-level programming language commonly used to add dynamic behavior and interactivity to web applications. It supports various programming concepts such as variables, arrays, functions, objects, classes, loops, and methods.

Arrays

An array is a collection of multiple values stored in a single variable. Array elements are accessed using an index, and the index starts from 0.

For example:

```
let marks = [75, 82, 68, 90, 85];
```

Here, 75 is at index 0, 82 is at index 1, and so on. JavaScript provides several built-in array methods:

- `push()` – adds an element to the end of an array.
- `pop()` – removes the last element.
- `shift()` – removes the first element.
- `unshift()` – adds an element to the beginning.
- `length` – returns the number of elements in an array.

Functions and Arrow Functions

A function is a reusable block of code designed to perform a particular task. Functions can accept parameters and can be called multiple times with different values. Arrow functions (`=>`) provide a shorter syntax for writing functions and are commonly used for simple, single-purpose operations.

In this experiment, functions are used to:

- Display all marks stored in an array.
- Find the highest mark in the array.
- Find the lowest mark in the array.
- Demonstrate arrow functions with no parameters, one parameter, and multiple parameters.

Encapsulation and Private Fields

Encapsulation is an object-oriented concept that restricts direct access to an object's internal data and allows it to be accessed only through defined methods. The `#` prefix is used to declare a private class field. A private field cannot be accessed directly from outside the class. In the `BankAccount` program, `#balance` is private and can only be read or modified through the `deposit()`, `withdraw()`, and `checkBalance()` methods.

Inheritance

Inheritance allows one class (child class) to acquire the properties and methods of another class (parent class) using the `extends` keyword. The `super()` function is used inside the child class constructor to call the constructor of the parent class. In this experiment, the `Manager` class inherits from the `Employee` class and adds its own property and method.

The basic concepts demonstrated in this experiment are:

Arrays → Array Methods → Functions → Arrow Functions → Encapsulation → Private Fields → Inheritance

5. ALGORITHM / PROCEDURE

1. Create a JavaScript project or folder.
2. Create the required JavaScript files.
3. Create an array containing initial marks.
4. Display the original array and its length using `console.log()`.
5. Access individual array elements using index numbers, including the last element.
6. Modify an element using its index.
7. Add an element using `push()` and remove it using `pop()`.
8. Add an element to the beginning using `unshift()` and remove it using `shift()`.
9. Create a function to display all elements of the array.

10. Create functions to find the highest and lowest values in the array.
11. Create arrow functions with no parameters, one parameter, and multiple parameters.
12. Create a BankAccount class with a private balance field.
13. Create a constructor to initialize the account balance.
14. Create methods to deposit, withdraw, and check the balance.
15. Create an Employee class with a constructor and a method to display employee details.
16. Create a Manager class that extends Employee using the extends and super keywords.
17. Add an additional property and method to the Manager class.
18. Execute the programs using a JavaScript runtime such as Node.js.
19. Observe and record the output of all three programs.

6. PROGRAM

Program 1: arr&fun.js

```
// Create an array
let marks = [75, 82, 68, 90, 85];
console.log("Original Array:", marks);
// 1. Length
console.log("Number of Elements:", marks.length);
// 2. Accessing Array Elements
console.log("First Mark:", marks[0]);
console.log("Third Mark:", marks[2]);
console.log("Last Mark:", marks[marks.length - 1]);
// 3. Changing an Element
marks[2] = 72;
console.log("After Changing Third Element:", marks);
// 4. push() - Add element at the end
marks.push(95);
console.log("After push():", marks);
// 5. pop() - Remove last element
marks.pop();
console.log("After pop():", marks);
// 6. unshift() - Add element at the beginning
marks.unshift(70);
console.log("After unshift():", marks);
// 7. shift() - Remove first element
marks.shift();
console.log("After shift():", marks);

// Function to display all elements
function showMarks(arr) {
  console.log("Student Marks:");
```

```

const welcome = () => {
  console.log("Welcome to JavaScript Arrays");
};
welcome();
// Arrow function with one parameter
const double = value => value * 2;

console.log("Double of 15:", double(15));
// Arrow function with multiple parameters
const multiply = (x, y) => {
  return x * y;
};
console.log("Multiplication of 6 and 7:", multiply(6, 7));

// Calling functions
showMarks(marks);
console.log("Highest Mark:", findHighest(marks));
console.log("Lowest Mark:", findLowest(marks));

    for (let i = 0; i < arr.length; i++) {
      console.log("Mark", i + 1, ":", arr[i]);
    }
  }
  // Function to find the highest mark
  ✓ function findHighest(arr) {
    let highest = arr[0];

    for (let i = 1; i < arr.length; i++) {
      if (arr[i] > highest) {
        highest = arr[i];
      }
    }

    return highest;
  }
  // Function to find the lowest mark
  ✓ function findLowest(arr) {
    let lowest = arr[0];

    for (let i = 1; i < arr.length; i++) {
      if (arr[i] < lowest) {
        lowest = arr[i];
      }
    }

    return lowest;
  }

```

Program 2: encapsulation.js

```

class BankAccount {
  #balance;

  constructor(initialBalance) {
    this.#balance = initialBalance;
  }

  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log("Amount deposited:", amount);
    } else {
      console.log("Invalid deposit amount");
    }
  }

  withdraw(amount) {
    if (amount > 0 && amount <= this.#balance) {
      this.#balance -= amount;
      console.log("Amount withdrawn:", amount);
    } else {
      console.log("Insufficient balance or invalid amount");
    }
  }

  checkBalance() {
    console.log("Current Balance:", this.#balance);
  }
}

// creating an object
let account = new BankAccount(5000);

// Accessing data through methods
account.checkBalance();

account.deposit(2000);
account.checkBalance();

account.withdraw(1500);
account.checkBalance();

```

Program 3: inheritance.js

```

class Employee {
  constructor(name, salary) {
    this.name = name;
    this.salary = salary;
  }
  displayEmployee() {
    console.log("Employee Name:", this.name);
    console.log("Salary:", this.salary);
  }
}
// Child class inherits from Employee
class Manager extends Employee {
  constructor(name, salary, department) {
    super(name, salary);
    this.department = department;
  }
  displayManager() {
    console.log("Department:", this.department);
  }
}

// Create object of child class
let manager1 = new Manager("Rahul", 50000, "IT");
// Calling inherited method
manager1.displayEmployee();
// Calling child class method
manager1.displayManager();

```

7. CODE EXPLANATION

Code / Concept	Explanation
let marks = [...]	Creates an array containing initial mark values.
marks.length	Returns the total number of elements in the array.
marks[0], marks[2]	Accesses elements at index 0 and index 2 of the array.
marks[marks.length - 1]	Accesses the last element of the array.
marks[2] = 72	Updates the element at index 2.
push()	Adds a new element to the end of an array.
pop()	Removes and returns the last element of an array.
unshift()	Adds a new element to the beginning of an array.
shift()	Removes and returns the first element of an array.
function showMarks()	Defines a function to display all marks in the array.

Code / Concept	Explanation
function findHighest()	Defines a function that returns the highest value in the array.
function findLowest()	Defines a function that returns the lowest value in the array.
const welcome = () => {}	Arrow function with no parameters.
const double = value => value * 2	Arrow function with a single parameter, written without braces.
const multiply = (x, y) => {}	Arrow function with multiple parameters.
class BankAccount	Defines a class named BankAccount.
#balance	Declares a private class field that cannot be accessed from outside the class.
constructor()	Initializes an object when it is created.
deposit()	Adds a valid amount to the private balance field.
withdraw()	Subtracts a valid amount from the private balance field.
checkBalance()	Displays the current value of the private balance field.
class Employee	Defines a parent class with name and salary properties.
class Manager extends Employee	Defines a child class that inherits from Employee.
super(name, salary)	Calls the parent class constructor from within the child class.
displayEmployee()	Inherited method that displays employee name and salary.
displayManager()	Method defined only in the Manager class to display the department.

8. TEST CASES AND EXECUTION DATA

Program 1: Array and Function Operations (arr&fun.js)

Test Case	Operation	Expected Result	Actual Result	Status
TC-01	Create marks array	Array should contain 75, 82, 68, 90, 85	Array created successfully	Pass
TC-02	Find array length	Length should be 5	Length displayed as 5	Pass
TC-03	Access index 0, 2 and last	75, 68 and 85 should be displayed	75, 68 and 85 displayed	Pass
TC-04	Update index 2	68 should change to 72	72 displayed	Pass
TC-05	Use push()	95 should be added at the end	95 added successfully	Pass
TC-06	Use pop()	95 should be removed	95 removed successfully	Pass

Test Case	Operation	Expected Result	Actual Result	Status
TC-07	Use unshift()	70 should be added at the beginning	70 added successfully	Pass
TC-08	Use shift()	70 should be removed	70 removed successfully	Pass
TC-09	Call showMarks()	All marks should be displayed with position	Marks displayed individually	Pass
TC-10	Call findHighest()	Highest mark should be 90	90 displayed	Pass
TC-11	Call findLowest()	Lowest mark should be 72	72 displayed	Pass
TC-12	Call welcome()	Welcome message should be displayed	Message displayed correctly	Pass
TC-13	Call double(15)	Result should be 30	30 displayed	Pass
TC-14	Call multiply(6, 7)	Result should be 42	42 displayed	Pass

Program 2: BankAccount Class – Encapsulation (encapsulation.js)

Test Case	Operation	Expected Result	Actual Result	Status
TC-15	Create BankAccount with 5000	Account should be created with balance 5000	Account created successfully	Pass
TC-16	Call checkBalance()	Balance should be 5000	5000 displayed	Pass
TC-17	Deposit 2000	Balance should increase to 7000	"Amount deposited: 2000" displayed	Pass
TC-18	Call checkBalance()	Balance should be 7000	7000 displayed	Pass
TC-19	Withdraw 1500	Balance should decrease to 5500	"Amount withdrawn: 1500" displayed	Pass
TC-20	Call checkBalance()	Balance should be 5500	5500 displayed	Pass

Program 3: Employee and Manager Classes – Inheritance (inheritance.js)

Test Case	Operation	Expected Result	Actual Result	Status
TC-21	Create Manager object	Object should inherit Employee properties	Object created successfully	Pass
TC-22	Call displayEmployee()	Name (Rahul) and salary (50000) should display	Employee details displayed	Pass
TC-23	Call displayManager()	Department (IT) should be displayed	"Department: IT" displayed	Pass

9. OUTPUT

Each of the three JavaScript programs was executed in Node.js, and each individual test case below was additionally run in isolation to capture its own console output, corresponding to the test cases in Section 8.

Program 1: Array and Function Operations (arr&fun.js)

Output 1 (TC-01): Create marks array

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc01.js
Original Array: [ 75, 82, 68, 90, 85 ]
|
```

Output 2 (TC-02): Find array length

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc02.js
Number of Elements: 5
|
```

Output 3 (TC-03): Access index 0, 2 and last

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc03.js
First Mark: 75
Third Mark: 68
Last Mark: 85
|
```

Output 4 (TC-04): Update index 2

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc04.js
After Changing Third Element: [ 75, 82, 72, 90, 85 ]
|
```

Output 5 (TC-05): Use push()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc05.js
After push(): [ 75, 82, 72, 90, 85, 95 ]
|
```

Output 6 (TC-06): Use pop()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc06.js
After pop(): [ 75, 82, 72, 90, 85 ]
|
```

Output 7 (TC-07): Use unshift()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc07.js
After unshift(): [ 70, 75, 82, 72, 90, 85 ]
|
```

Output 8 (TC-08): Use shift()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc08.js
After shift(): [ 75, 82, 72, 90, 85 ]
|
```

Output 9 (TC-09): Call showMarks()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc09.js
Student Marks:
Mark 1 : 75
Mark 2 : 82
Mark 3 : 72
Mark 4 : 90
Mark 5 : 85
|
```

Output 10 (TC-10): Call findHighest()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc10.js
Highest Mark: 90
|
```

Output 11 (TC-11): Call findLowest()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc11.js
Lowest Mark: 72
|
```

Output 12 (TC-12): Call welcome()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc12.js
Welcome to JavaScript Arrays
|
```

Output 13 (TC-13): Call double(15)

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc13.js
Double of 15: 30
|
```

Output 14 (TC-14): Call multiply(6, 7)

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc14.js
Multiplication of 6 and 7: 42
|
```

Program 2: BankAccount Class – Encapsulation (encapsulation.js)

Output 15 (TC-15): Create BankAccount with 5000

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc15.js
Account created successfully.
Current Balance: 5000
|
```

Output 16 (TC-16): Call checkBalance()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc16.js
Current Balance: 5000
|
```

Output 17 (TC-17): Deposit 2000

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc17.js
Amount deposited: 2000
|
```

Output 18 (TC-18): Call checkBalance()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc18.js
Current Balance: 7000
|
```

Output 19 (TC-19): Withdraw 1500

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc19.js
Amount withdrawn: 1500
|
```

Output 20 (TC-20): Call checkBalance()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc20.js
Current Balance: 5500
|
```

Program 3: Employee and Manager Classes – Inheritance (inheritance.js)

Output 21 (TC-21): Create Manager object

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc21.js  
Manager object created: true  
|
```

Output 22 (TC-22): Call displayEmployee()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc22.js  
Employee Name: Rahul  
Salary: 50000  
|
```

Output 23 (TC-23): Call displayManager()

```
PS C:\Users\anits-csm\Desktop\csmb99> node tc23.js  
Department: IT  
|
```

10. RESULT

Thus, the basic concepts of JavaScript arrays, functions, and object-oriented programming were successfully implemented and executed. Array operations such as indexing, updating, push(), pop(), shift(), and unshift() were demonstrated, along with custom functions to find the highest and lowest marks and arrow functions with varying numbers of parameters. Encapsulation was implemented using a private class field (#balance) accessed only through public methods, and inheritance was demonstrated using the extends and super keywords in the Employee and Manager classes. All three programs were executed successfully and the expected outputs were obtained, with individual outputs captured for each test case above.